SENG 225 | DATA STRUCTURES AND ALGORITHMS

Lab handout and instructor answer guide | C programming | 90 minutes

Topics: pointers, arrays, structures, singly linked lists, dynamic memory management, and time complexity.

Learning outcomes

- Distinguish pointer addresses from the values stored at those addresses.
- Manipulate arrays and structures through pointers.
- Build, traverse, modify, and release a singly linked list.
- Recognize edge cases and assess basic time and auxiliary-space complexity.

Lab schedule

00-10 min | Pointer-based swap

10-25 min | Reverse an array with two pointers

25-40 min | Student records using struct

40-60 min | Create and traverse a linked list

60-80 min | Insert and delete nodes

80-90 min | Independent practice and discussion

1 | Pointer-based swap (10 min)

Task: Exchange two integer values using a function. Begin by asking students to predict whether a function taking plain int arguments could modify the caller's variables.

Source file: 01_pointer_swap.c

```
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
/* Call: swap(&x, &y); */
```

Example output: Before: x = 10, y = 20 | After: x = 20, y = 10

Question: What is the difference between &x and *a?

Answer: &x evaluates to the address of x. *a dereferences pointer a and accesses the int stored at the address it points to. When a points to x, modifying *a modifies x.

Question: What happens if we call swap(x, y) rather than swap(&x, &y)?

Answer: The call is invalid for the declared function: swap expects int * arguments, while x and y are int values. A compiler diagnostic is required. Changing the function to take int parameters would compile, but only local copies would be exchanged; the caller's variables would remain unchanged.

Question: What is the time complexity of swap?

Answer: O(1) time and O(1) auxiliary space: it performs a constant number of assignments.

2 | Reverse an array in place (15 min)

Task: Reverse {10, 20, 30, 40, 50} without allocating a second array. Use a left pointer and a right pointer and swap the indicated values.

Source file: 02_reverse_array.c

```
int *left = arr;
int *right = arr + size - 1;
while (left < right) {
    int temp = *left;
    *left++ = *right;
    *right-- = temp;
}
```

Expected output: 50 40 30 20 10. The supplied function checks size <= 1 before computing the right pointer.

Question: How is arr[i] related to *(arr + i)?

Answer: For an ordinary array and a valid index i, arr[i] is defined equivalently to *(arr + i). Pointer arithmetic advances in units of sizeof(*arr), not individual bytes.

Question: What is the time complexity?

Answer: O(n): about floor(n/2) swaps are performed for an array containing n elements. Each swap takes constant time.

Question: What is the auxiliary-space complexity?

Answer: O(1): the algorithm stores only a few pointers and one temporary value, independent of array length.

Question: What should happen with an empty or single-element array?

Answer: There are no pairs to exchange, so the function should return immediately. The example guards size <= 1 to avoid forming arr - 1 for a zero-length array.

3 | Student records with struct (15 min)

Task: Store student IDs, names, and grades in an array of structures, then find the student with the highest grade.

Source file: 03_student_struct.c

```
struct Student {
    int id;
    char name[30];
    float grade;
};
struct Student *ptr = &students[0];
printf("%s / %s\n", ptr->name, (*ptr).name);
```

Expected highest grade: Ayse (92.0).

Question: What is the difference between . and ->?

Answer: The dot operator accesses a field through a structure object (student.grade). The arrow operator accesses a field through a pointer to a structure (ptr->grade). ptr->grade is shorthand for (*ptr).grade.

Question: Why are parentheses required in (*ptr).name?

Answer: The . operator binds more tightly than unary *. Without parentheses, *ptr.name tries to access name on ptr itself, which is a pointer, and is invalid.

Question: What is the time complexity for finding the highest grade?

Answer: O(n) time from scanning n student records; O(1) extra space to retain the index of the best record.

4 | Build and traverse a singly linked list (20 min)

Task: Create nodes containing 10, 20, and 30; append them to a list; print the list; and release the allocated memory.

Source file: 04_linked_list_basics.c

```
struct Node {
    int data;
    struct Node *next;
};
/* Example list: head -> [10|next] -> [20|next] -> [30|NULL] */
```

Expected output: 10 -> 20 -> 30 -> NULL

Live coding order: define Node; createNode with malloc error handling; insertEnd; printList; freeList. For each pointer modification, trace the head and next links on the board.

Question: Why does insertEnd receive struct Node **head rather than struct Node *head?

Answer: When the list is initially empty, insertEnd must update the caller's head variable. Passing its address allows the function to write *head = newNode. A single pointer passed by value would allow changing nodes but not replacing the caller's head pointer.

Question: Why is the last node's next field set to NULL?

Answer: NULL marks the end of the linked list. Traversal continues while the current node pointer is not NULL; missing termination can cause invalid memory access or endless traversal.

Question: What is malloc used for, and why check its return value?

Answer: malloc requests heap storage whose lifetime extends beyond the creating function. It can return NULL on allocation failure; dereferencing NULL is invalid, so check before assigning fields.

Question: Why must allocated nodes be freed?

Answer: Each successful malloc allocation must eventually be released when no longer needed. Otherwise the program leaks memory. Save the next pointer before freeing the current node, since accessing a freed node is invalid.

Question: What is the cost of insertEnd in this implementation?

Answer: O(n) time for a non-empty list because the function walks from head to the last node. Keeping a tail pointer would make append O(1), assuming the tail is maintained correctly.

5 | Insert and delete nodes (20 min)

Task: Add 5 at the beginning, then remove the first node containing 20. Also test removing the head, missing values, and an empty list.

Source file: 05_linked_list_insert_delete.c

```
void insertBeginning(struct Node **head, int value) {
    struct Node *node = createNode(value);
    node->next = *head;
    *head = node;
}
```

Expected sequence: 10 -> 20 -> 30 -> NULL; then 5 -> 10 -> 20 -> 30 -> NULL; then 5 -> 10 -> 30 -> NULL.

Question: What changes when deleting the head node?

Answer: There is no previous node. Update *head to the old head's next pointer and then free the old head. For an interior node, update previous->next instead.

Question: What happens if the value is absent or the list is empty?

Answer: The search reaches NULL without finding a match, and the implementation returns 0 without changing the list. The sample program prints Value not found! for this result.

Question: What if the list contains duplicate values?

Answer: The demonstrated deleteValue implementation removes only the first matching node. Deleting all matches requires continuing the traversal and carefully maintaining links.

Question: Why should free(current) happen only after relinking?

Answer: The next pointer must be preserved through the list link update before current is deallocated. After free(current), reading current->next would be a use-after-free error.

Question: What is the complexity of insertBeginning and deleteValue?

Answer: insertBeginning is O(1) time; deleteValue is O(n) in the worst case because it searches for a value. Both use O(1) auxiliary space.

6 | Independent student exercises (10 min)

Student starter: 06_student_exercises.c. Instructor solutions: 07_exercise_solutions.c (keep private until students finish).

Count Nodes

Task: Implement int countNodes(const struct Node *head).

Expected: 10 -> 20 -> 30 -> NULL => 3

Instructor answer: Traverse once and increment a counter for each node; return 0 for an empty list. O(n) time, O(1) extra space.

Search a Value

Task: Implement int search(const struct Node *head, int value).

Expected: Search 20 in 10 -> 20 -> 30 => 1; search 99 => 0

Instructor answer: Walk through each node and compare data to value; return 1 at first match or 0 at NULL. O(n) worst-case time.

Find Maximum

Task: Implement int findMax(const struct Node *head) for a non-empty list.

Expected: 10 -> 50 -> 20 => 50

Instructor answer: Initialize the maximum to head->data and scan remaining nodes. For an empty list, define a separate error contract; this exercise explicitly requires a non-empty list.

Reverse Linked List

Task: Implement void reverseList(struct Node **head) without allocating nodes.

Expected: 10 -> 20 -> 30 -> NULL => 30 -> 20 -> 10 -> NULL

Instructor answer: Use previous, current, and next pointers. Save current->next, redirect current->next to previous, then advance. At completion assign *head = previous. O(n) time, O(1) extra space.

7 | Complexity discussion

Operation	Array	Singly linked list
Indexed access	O(1)	O(n)
Prepend	O(n) (shift elements)	O(1)

Append	O(1) amortized*	O(n) with head only
Search by value	O(n)	O(n)
Delete after locating predecessor	O(n) (shift elements)	O(1) link update

* Array append assumes a dynamic array with spare capacity and geometric growth; a fixed-capacity array may have a different contract. Linked-list deletion assumes the predecessor is already known; locating it is a separate O(n) operation.

Question: Is a linked list always more efficient than an array?

Answer: No. A linked list supports constant-time local pointer updates, but it has O(n) indexed access, per-node allocation overhead, and weaker memory locality. Arrays often offer faster iteration and O(1) indexed access.

Question: What common pointer bugs should we watch for?

Answer: Uninitialized pointers, NULL dereferences, losing the head pointer, forming broken links, memory leaks, double free, and use-after-free. Test empty and one-node lists explicitly.

Teaching checklist / exit ticket

- Can you explain what * and & do in the swap example?
- Can you implement one two-pointer array algorithm?
- Can you explain why ptr->field equals (*ptr).field?
- Can you draw the head/next links before and after insertion and deletion?
- Can you state the time cost of traversal, prepend, and append with head only?

Instructor note: The standalone source files, complete programs, and the answer implementations are provided in the separate ZIP. Compile programs individually; every source file has its own main().